Iteration

Tad Dallas

Contents

What is iteration?

Iteration refers to the process of doing the same task to a bunch of different objects. Consider a toy example of the actions required by a cashier at a grocery store. They scan each item, where items can be different sizes/shapes/prices. This is an iterative task, as it uses the same motions (essentially) across a variety of different objects (groceries) which may vary in many ways, but have some commonalities (e.g., most items have a barcode).

Why is iteration important?

Up until this point, we have dealt with single data.frame objects (or vectors, the building blocks of data.frames). However, we also introduced the concept of `lists` in one of the first lectures, and will go into more detail about lists soon. For now, we'll talk about iteration independent of list objects, but keep in mind that iteration is important for lists.

Essentially, iteration allows us to process a large amount of data without the need to repeat ourselves. Recall the gapminder data.

```
dat <- read.delim(file = "http://www.stat.ubc.ca/~jenny/notOcto/STAT545A/examples/gapminder/data/gapmin
```

We discussed the `gapminder` data when introducing some tools around data subsetting and summarising. We ended that lecture by discussing `dplyr`, a useful package for data processing.

```
library(plyr)
library(dplyr)
```

Recall that towards the end of that lecture, we introduce piping commands with `dplyr` to summarise data. For instance, the code below calculates mean life expectancy (`lifeExp`) by `country`.

```
tmp3 <- dat |>
    dplyr::group_by(country) |>
    dplyr::summarise(mnLifeExp=mean(lifeExp))
```

Approaching this with `dplyr` offers us a powerful way to summarise our data, but you will inevitably hit the limits of `dplyr` and thinking about how to do this in base R is difficult, right? In base R, we discussed subsetting, but to do what the above code does, we would have to subset by every country and then calculate the mean `lifeExp` for each subset. This is a good jumping off point for iteration, starting with the idea of the `for` loop (some folks use 'looping' and 'iteration' to mean the same thing). So we want a way to subset the `dat` data.frame by country, and then calculate mean `lifeExp`.

To start, we need to get a vector of the countries in the data.

```
countries <- unique(dat$country)
```

Then we need to get the overall structure of the loop in place. To do this, we use the structure `for(i in range){ do something}`. Essentially, we need to first define the range of what we want the loop to do, and then within the curly brackets, we need to do the thing. The power of this comes from the `i` in the `for` loop call. This is essentially saying to temporally treat `i` as one of the values in `range`, do something considering that, and then set `i` to the next value. This sequential process means that at the end of the loop, we will have cycled through all the entries in `range`.

```
for(i in countries){
  print(i)
}
```

```
## [1] "Afghanistan"
## [1] "Albania"
## [1] "Algeria"
## [1] "Angola"
## [1] "Argentina"
## [1] "Australia"
## [1] "Austria"
## [1] "Bahrain"
## [1] "Bangladesh"
## [1] "Belgium"
## [1] "Benin"
## [1] "Bolivia"
## [1] "Bosnia and Herzegovina"
## [1] "Botswana"
## [1] "Brazil"
## [1] "Bulgaria"
## [1] "Burkina Faso"
## [1] "Burundi"
## [1] "Cambodia"
## [1] "Cameroon"
## [1] "Canada"
## [1] "Central African Republic"
## [1] "Chad"
## [1] "Chile"
## [1] "China"
## [1] "Colombia"
## [1] "Comoros"
## [1] "Congo, Dem. Rep."
## [1] "Congo, Rep."
## [1] "Costa Rica"
## [1] "Cote d'Ivoire"
## [1] "Croatia"
## [1] "Cuba"
## [1] "Czech Republic"
## [1] "Denmark"
## [1] "Djibouti"
## [1] "Dominican Republic"
## [1] "Ecuador"
## [1] "Egypt"
## [1] "El Salvador"
## [1] "Equatorial Guinea"
```

```
## [1] "Eritrea"
## [1] "Ethiopia"
## [1] "Finland"
## [1] "France"
## [1] "Gabon"
## [1] "Gambia"
## [1] "Germany"
## [1] "Ghana"
## [1] "Greece"
## [1] "Guatemala"
## [1] "Guinea"
## [1] "Guinea-Bissau"
## [1] "Haiti"
## [1] "Honduras"
## [1] "Hong Kong, China"
## [1] "Hungary"
## [1] "Iceland"
## [1] "India"
## [1] "Indonesia"
## [1] "Iran"
## [1] "Iraq"
## [1] "Ireland"
## [1] "Israel"
## [1] "Italy"
## [1] "Jamaica"
## [1] "Japan"
## [1] "Jordan"
## [1] "Kenya"
## [1] "Korea, Dem. Rep."
## [1] "Korea, Rep."
## [1] "Kuwait"
## [1] "Lebanon"
## [1] "Lesotho"
## [1] "Liberia"
## [1] "Libya"
## [1] "Madagascar"
## [1] "Malawi"
## [1] "Malaysia"
## [1] "Mali"
## [1] "Mauritania"
## [1] "Mauritius"
## [1] "Mexico"
## [1] "Mongolia"
## [1] "Montenegro"
## [1] "Morocco"
## [1] "Mozambique"
## [1] "Myanmar"
## [1] "Namibia"
## [1] "Nepal"
## [1] "Netherlands"
## [1] "New Zealand"
## [1] "Nicaragua"
## [1] "Niger"
## [1] "Nigeria"
```

```
## [1] "Norway"
## [1] "Oman"
## [1] "Pakistan"
## [1] "Panama"
## [1] "Paraguay"
## [1] "Peru"
## [1] "Philippines"
## [1] "Poland"
## [1] "Portugal"
## [1] "Puerto Rico"
## [1] "Reunion"
## [1] "Romania"
## [1] "Rwanda"
## [1] "Sao Tome and Principe"
## [1] "Saudi Arabia"
## [1] "Senegal"
## [1] "Serbia"
## [1] "Sierra Leone"
## [1] "Singapore"
## [1] "Slovak Republic"
## [1] "Slovenia"
## [1] "Somalia"
## [1] "South Africa"
## [1] "Spain"
## [1] "Sri Lanka"
## [1] "Sudan"
## [1] "Swaziland"
## [1] "Sweden"
## [1] "Switzerland"
## [1] "Syria"
## [1] "Taiwan"
## [1] "Tanzania"
## [1] "Thailand"
## [1] "Togo"
## [1] "Trinidad and Tobago"
## [1] "Tunisia"
## [1] "Turkey"
## [1] "Uganda"
## [1] "United Kingdom"
## [1] "United States"
## [1] "Uruguay"
## [1] "Venezuela"
## [1] "Vietnam"
## [1] "West Bank and Gaza"
## [1] "Yemen, Rep."
## [1] "Zambia"
## [1] "Zimbabwe"
```

So what did the above code do?

Alright. So we have a way to sequentially work through all of the `countries` and we know how to subset the data based on country. So we can now subset the data for each of the countries, using the `i` iterator as a stand-in for each of the country names. But this does not actually do anything with the data, such that `tmp` will just be the subset data for the last country in the `countries` vector.

```
for(i in countries){
  tmp <- dat[which(dat$country == i), ]
}
```

So let's now compute the mean `lifeExp` for each country.

```
meanLifeExp <- c()
for(i in countries){
  tmp <- dat[which(dat$country == i), ]
  meanLifeExp <- c(meanLifeExp, mean(tmp$lifeExp))
}
```

Here, we first create a vector to hold the output data (`meanLifeExp`) and then append the value for each mean onto the vector. That is, we essentially re-write the `meanLifeExp` vector at every step of the iteration. This is bad practice for a number of reasons (e.g., no memory efficient, writing over objects where the object itself is in the call is bad practice, etc.). So how can we get around doing this? `for` loops can be handed a vector of character values (as we have done above) or they can be handed a numeric range. This is often useful, as it eases indexing and can be a bit clearer in the code.

```
meanLifeExp <- c()
for(i in 1:length(countries)){
  tmp <- dat[which(dat$country == countries[i]), ]
  meanLifeExp[i] <- mean(tmp$lifeExp)
}
```

And the results of this code should be the same as the other `for` loop. We now have a vector of mean life expectancy values for each country in `countries`. But that was a fair bit of work to get the same thing we could have gotten with `dplyr`, right? Let's explore a situation where it would be a bit tougher to get the same thing out of `dplyr` (at least with our current knowledge, as the example I'll give below can be solved using `dplyr::do`).

Let's say that we want to explore the relationship between `year` and `lifeExp` for each country. That is, we want to know how life expectancy is changing over time across the different countries. To do this, we can use the `cor.test` function in R to calculate Pearson's correlation coefficients (assumes linear structure between the two variables) or Spearman's rank correlation (assumes monotonic, but not linear response). The output of `cor.test` is a object, such that `dplyr::summarise` would fail.

```
tmp3 <- dat |>
    dplyr::group_by(country) |>
    dplyr::summarise(cor.test(year, lifeExp))
```

So `summarise` expects the output to be a vector (note that there are ways around this, by pulling out the information we want from the cor.test)

```
tmp3 <- dat |>
    dplyr::group_by(country) |>
    dplyr::summarise(cor.test(year, lifeExp)$estimate)
```

But how we do pull out multiple values from the same test? And how do we handle and diagnose potential errors when we don't work through each test sequentially?

```
lifeExpTime <- matrix(0, ncol=4, nrow=length(countries))

for(i in 1:length(countries)){
  tmp <- dat[which(dat$country == countries[i]), ]
  crP <- cor.test(tmp$year, tmp$lifeExp)
  crS <- cor.test(tmp$year, tmp$lifeExp, method='spearman')
  lifeExpTime[i, ] <- c(crP$estimate, crP$p.value,
```

```
    crS$estimate, crS$p.value)
}
```

```
## Warning in cor.test.default(tmp$year, tmp$lifeExp, method = "spearman"): Cannot
## compute exact p-value with ties
```

```
colnames(lifeExpTime) <- c('pearsonEst', 'pearsonP',
  'spearmanEst', 'spearmanP')

lifeExpTime <- as.data.frame(lifeExpTime)
lifeExpTime$country <- countries
```

And we can explore these data, to determine which countries have increasing or decreasing life expectancy values as a function of time.

```
lifeExpTime[which.min(lifeExpTime$pearsonEst),]
```

```
##     pearsonEst  pearsonP spearmanEst spearmanP country
## 141 -0.2446149 0.4435318  -0.1888112 0.5578278  Zambia
```

This may seem like a lot of work when we could have done a bit less using `dplyr` syntax. The real power of `for` loops will be in working with lists, simulating data, and plotting. For instace, let's say we don't have data directly to work with, but want to generate data. We could generate a bunch of data, mash it all together in a data.frame, and then feed it into `dplyr`, the data generation step would require a `for` loop already, so why not keep things all contained in the `for` loop.

Let's say we want to create a Fibonacci sequence. This is a vector of numbers in which each number is the sum of the two preceding numbers in the vector. For the example, we will limit the length of the vector to be length 1000.

```
fib <- c(0,1)
for(i in 3:1000){
  fib[i] <- sum(fib[(i-2):(i-1)])
}
```

And now we have a Fibonacci sequence starting with `c(0,1)`.

> Why do I start the `for` loop above at 3, and how else could you approach this same problem (there are many ways)?

```
# You do it.
```

Apply statements

`apply` statements exist in many types, depending on the data.structure you wish to do the action on: e.g. `apply`, `sapply`, `lapply`, `vapply`, `tapply`. We will focus on `apply` and `lapply`, but realize that these other options may be better suited for your use case (especially `vapply`, which gives you a bit more control over output format). In the loop above, we wanted to find the mean of each entry in a list. We used a `for` loop to loop over elements, and stored the resulting means in a vector called `out`. Instead, we could use `lapply`... the `l` in it means it performs some action on a list object.

We will use biological data on ringtail poop sampled between 2020 and 2022 in Zion National Park and Grand Canyon National Park by Anna Willoughby. More information on the goals of the project are available here (https://github.com/DrakeLab/willoughby-ringtail-fieldwork).

```
dat2 <- read.csv("https://raw.githubusercontent.com/DrakeLab/willoughby-ringtail-fieldwork/refs/heads/ma

dat2 <- dat2[,1:10 ]
```

```
testList2 <- split(dat2, dat2$FragmentType)
```

The `split` function takes a data.frame and splits it into a list of data.frames, where each data.frame is a subset of the original data.frame. The `lapply` function takes a list and applies a function to each element in the list.

```
lapply(X=testList2, FUN=nrow)
```

```
## $Anthropogenic
## [1] 107
##
## $Contaminant
## [1] 22
##
## $Invertebrate
## [1] 41
##
## $Organic
## [1] 2
##
## $Plant
## [1] 51
##
## $Unknown
## [1] 18
##
## $Vertebrate
## [1] 127
```

The output of `lapply` will always be a list, which is nice in some instances and not nice in others. `sapply` is a wrapper for `lapply` which always returns a vector of values.

```
sapply(X=testList2, FUN=nrow)
```

```
## Anthropogenic   Contaminant  Invertebrate       Organic         Plant
##           107            22            41             2            51
##       Unknown    Vertebrate
##            18           127
```

Now that we have an idea of what the `apply` family of functions do, we can look specifically at `apply`, which operates on matrices or data.frames. What if we wanted to calculate the mean of every column or row in a data.frame? We could loop over each column or row...

```
testDF <- data.frame(a=runif(100), b=rpois(100,2), d=rbinom(100,1,0.5))


# over columns
ret <- c()
for(i in 1:ncol(testDF)){
    ret[i] <- mean(testDF[,i])
}


# over rows
ret <- c()
for(i in 1:nrow(testDF)){
```

```
    ret[i] <- mean(unlist(testDF[i, ]))
}
```

Or we could use apply statements

```
apply(X=testDF, MARGIN=2, FUN=mean)
```

```
##         a         b         d
## 0.4795323 2.0300000 0.5000000
```

```
apply(X=testDF, MARGIN=1, FUN=mean)
```

```
##   [1] 1.1171007 0.8626835 0.8055115 1.7634570 1.7220546 1.7598058 0.4476039
##   [8] 1.2442895 0.6731413 2.6034845 1.0215511 0.5772091 2.0245974 0.5994766
##  [15] 0.4490901 1.6132605 1.0675513 1.1498955 1.2563169 0.4675040 1.6128144
##  [22] 0.9791861 2.5567006 1.1363490 1.2966616 2.1691176 0.6571939 1.5562721
##  [29] 1.0409537 0.7789659 0.6842775 0.1031870 0.5559534 1.6174360 0.4292563
##  [36] 1.1271428 0.7116007 1.0478829 0.5235936 1.2156014 0.3451333 0.8947058
##  [43] 1.7804579 0.6822583 0.7209590 1.1814663 0.5345600 1.0289597 0.5047802
##  [50] 0.1452886 0.9746693 0.6127967 0.8958768 1.2802492 0.3761445 1.0284006
##  [57] 0.5138722 1.0286871 1.1175946 0.5650631 0.5564265 0.9991473 1.1117179
##  [64] 1.4448111 1.0311504 1.0207782 0.5535076 0.2794253 1.2987800 0.9643015
##  [71] 1.1677552 1.0758149 1.0430423 1.1110099 0.4026340 1.3195719 0.2639175
##  [78] 1.4295455 1.7600145 1.6198344 1.2346495 1.6529415 1.4011285 0.3436138
##  [85] 0.8888720 1.7983354 1.3257291 1.2845489 0.6533702 1.1725225 0.3938764
##  [92] 0.9671779 0.4850271 0.3457208 1.0601237 1.3527537 0.4435835 0.4025798
##  [99] 0.1727447 1.2476045
```

One advantage is that indexing rows of a data.frame is a pain, which is why we had to `unlist` each row in the for loop over rows above. If we do not do this, we get a vector of NA values. This is because a data.frame is a list of vectors. This is why column-wise operations on data.frames can also be performed using `lapply` (if we wanted list output) or `sapply` (if we wanted vector output).

```
lapply(X=testDF, FUN=mean)
```

```
## $a
## [1] 0.4795323
##
## $b
## [1] 2.03
##
## $d
## [1] 0.5
```

```
sapply(X=testDF, FUN=mean)
```

```
##         a         b         d
## 0.4795323 2.0300000 0.5000000
```

Some practice problems

1. Above, we defined a data set as a list on ringtail diet changes as a function of time. Using those data, calculate the mean dry fragment weight for each of the fragment types.
2. The data are divided out into `Segments` and `Fragments`, where `Segments` are composed on smaller fragments of different types. Using what you know (either for loop or apply style statements), calculate the fraction of `Anthropogenic` fragments (`Anthropogenic` is a `FragmentType`) in each `Segment`.

A fun class exercise

You are creating a game of rock-paper-scissors. In the game, each player can select their strategy, and the strategy can be different in each trial (where there can be 100s of trials).

I think that the outcome is random, so as a player, I already have decided what I'm going to play before the game starts.

```
strat <- sample(c('rock','paper', 'scissors'), 100, replace=TRUE)
```

Write a for loop to simulate rock-paper-scissors game of 500 trials between two players, where my strategy above is one of the players.

```
newStrat <- c()
for(i in 1:length(strat)){
  newStrat[i] <- sample(c('rock','paper','scissors'),1)
}
```

But this is just the same as above. And we need a way to score the result to see who won right? Let's do that now. Write a function that determines who wins each round (so the inputs would be something like x='rock',y='scissors', and it would output `y` or `2` to indicate the winner)

```
getScore <- function(x,y){
  xScore <- yScore <- c()
  payoff <- matrix(c(0,1,0,0,0,1,1,0,0), ncol=3, byrow=TRUE)
  colnames(payoff) <- rownames(payoff) <- c('rock', 'scissors', 'paper')
  for(i in 1:length(x)){
    xScore[i] <- payoff[which(rownames(payoff) == x[i]), which(colnames(payoff)==y[i])]
    yScore[i] <- payoff[which(rownames(payoff) == y[i]), which(colnames(payoff)==x[i])]
  }
  return(c(x=sum(xScore), y=sum(yScore)))
}
```

How would you go about changing the strategy of the other player to beat my strategy? I'm basically asking you to write code to play the winning move against the opponent when you know the opponent's strategy.

```
tadStrat <- function(opp){
  opts <- c('rock', 'paper', 'scissors')
  ret <- sample(opts, 1)
  for(i in 2:length(opp)){
    if(opp[i-1] == 'rock'){
      ret[i] <- 'scissors'
    }
    if(opp[i-1] == 'paper'){
      ret[i] <- 'rock'
    }
    if(opp[i-1] == 'scissors'){
      ret[i] <- 'paper'
    }
  }
  return(ret)
}
```

Let's use apply/lapply/etc some more. Let's say that I do not allow you to see my strategy directly, but I do let you test any number of strategies against mine and use the best one. Write code to generate many different strategies and select the best one given my fixed strategy `strat`.

```
stratList <- lapply(1:100000, function(x){
  strat <- sample(c('rock','paper', 'scissors'), 100, replace=TRUE)
})

stratOut <- lapply(stratList, function(x){
  getScore(x, strat)
})

scoreDiff <- sapply(stratOut, function(x){x[1]-x[2]})
stratOut[[which.max(scoreDiff)]]
```

```
##  x  y
## 58 17
```

```
stratList[[which.max(scoreDiff)]]
```

```
##   [1] "paper"    "rock"     "rock"     "paper"    "rock"     "scissors"
##   [7] "paper"    "rock"     "rock"     "scissors" "rock"     "rock"
##  [13] "scissors" "scissors" "scissors" "scissors" "scissors" "rock"
##  [19] "paper"    "paper"    "rock"     "scissors" "scissors" "scissors"
##  [25] "paper"    "rock"     "rock"     "rock"     "paper"    "rock"
##  [31] "rock"     "rock"     "paper"    "rock"     "scissors" "paper"
##  [37] "scissors" "rock"     "paper"    "paper"    "rock"     "rock"
##  [43] "rock"     "scissors" "rock"     "scissors" "scissors" "rock"
##  [49] "scissors" "scissors" "rock"     "rock"     "rock"     "rock"
##  [55] "scissors" "rock"     "scissors" "rock"     "paper"    "paper"
##  [61] "scissors" "scissors" "paper"    "rock"     "rock"     "scissors"
##  [67] "paper"    "scissors" "scissors" "scissors" "paper"    "rock"
##  [73] "paper"    "rock"     "rock"     "scissors" "paper"    "paper"
##  [79] "rock"     "rock"     "rock"     "scissors" "scissors" "rock"
##  [85] "rock"     "scissors" "rock"     "rock"     "rock"     "rock"
##  [91] "paper"    "rock"     "paper"    "paper"    "rock"     "scissors"
##  [97] "paper"    "paper"    "rock"     "paper"
```

What other way could we approach this problem?

```
# a bit obvious, but I can break down each play and test all 3 strategies, picking the one that works b

ret <- c()
for(i in 1:length(strat)){
  if(getScore('rock', strat[i])[1] == 1){
    ret[i] <- 'rock'
  }
  if(getScore('scissors', strat[i])[1] == 1){
    ret[i] <- 'scissors'
  }
  if(getScore('paper', strat[i])[1] == 1){
    ret[i] <- 'paper'
  }
}
```

sessionInfo

```
sessionInfo()
```

```
## R version 4.5.0 (2025-04-11)
```

```
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 22.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/atlas/libblas.so.3.10.3
## LAPACK: /usr/lib/x86_64-linux-gnu/atlas/liblapack.so.3.10.3;  LAPACK version 3.10.0
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: America/New_York
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] tidyr_1.3.1    dplyr_1.1.4    plyr_1.8.9     DBI_1.2.3      geodata_0.5-8
##  [6] terra_1.8-50   sf_1.0-21      rgbif_3.7.7    jsonlite_2.0.0 httr_1.4.7
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.6       xfun_0.52          ggplot2_3.5.2      vctrs_0.6.5
##  [5] tools_4.5.0        generics_0.1.4     curl_6.3.0         tibble_3.3.0
##  [9] proxy_0.4-27       RSQLite_2.3.7      blob_1.2.4         pkgconfig_2.0.3
## [13] KernSmooth_2.23-26 data.table_1.17.6  dbplyr_2.5.0       RColorBrewer_1.1-3
## [17] lifecycle_1.0.4    compiler_4.5.0     farver_2.1.2       stringr_1.5.1
## [21] tinytex_0.57       codetools_0.2-19   htmltools_0.5.8.1  maps_3.4.1
## [25] class_7.3-23       yaml_2.3.10        lazyeval_0.2.2     pillar_1.10.2
## [29] whisker_0.4.1      classInt_0.4-11    cachem_1.1.0       tidyselect_1.2.1
## [33] digest_0.6.37      stringi_1.8.7      purrr_1.0.4        fastmap_1.2.0
## [37] grid_4.5.0         cli_3.6.5          magrittr_2.0.3     utf8_1.2.6
## [41] triebeard_0.4.1    crul_1.5.0         e1071_1.7-16       withr_3.0.2
## [45] scales_1.4.0       bit64_4.6.0-1      oai_0.4.0          rmarkdown_2.29
## [49] bit_4.6.0          memoise_2.0.1      evaluate_1.0.4     knitr_1.50
## [53] rlang_1.1.6        urltools_1.7.3     Rcpp_1.0.14        glue_1.8.0
## [57] httpcode_0.3.0     xml2_1.3.8         R6_2.6.1           units_0.8-7
```